Lab7

Course: ARTI 404 – Image Processing

Student Name: Muhannad Almutlaq

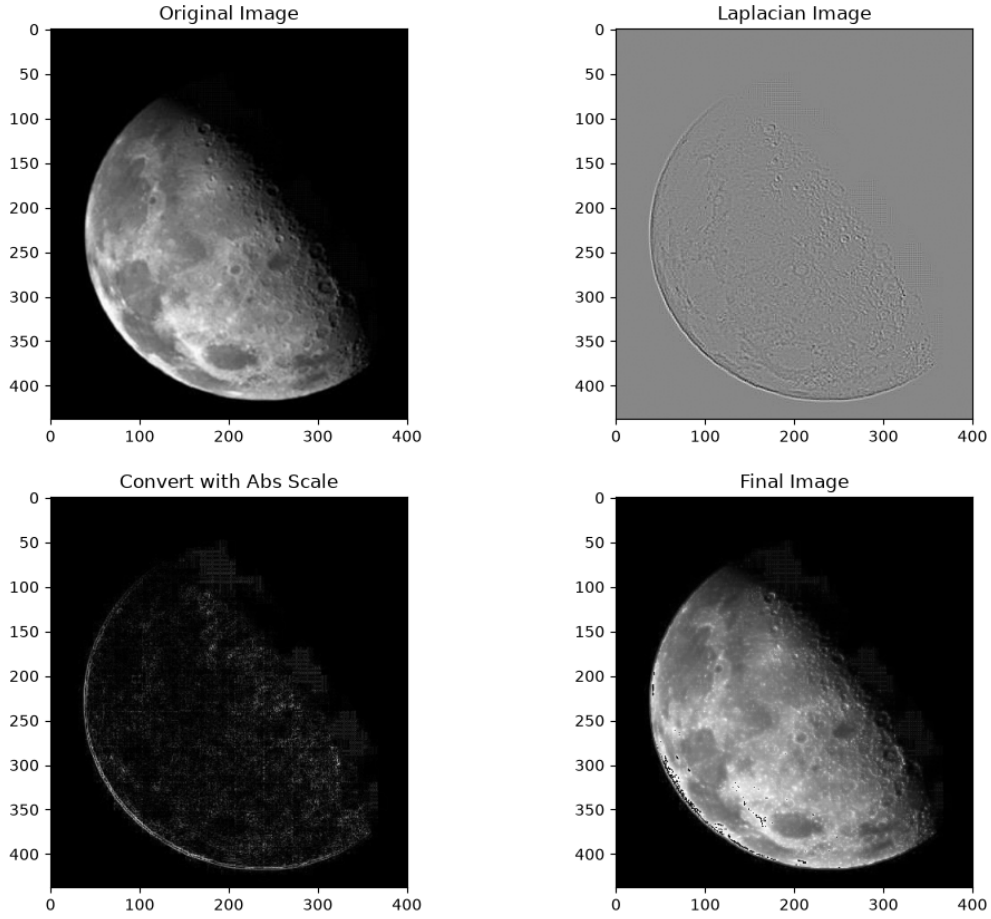
Student ID: 2240006060

Procedural Task#1

Apply a Laplacian filter with a manually defined kernel to sharpen the moon image. Load moon.jpg in grayscale and define the standard 3x3 Laplacian kernel [[0, 1, 0], [1, -4, 1], [0, 1, 0]], which detects horizontal and vertical edges (the alternative 8-neighbour kernel also detects diagonal edges). Apply it with cv2.filter2D using cv2.CV_64F so negative values are not clipped, convert the result to 8-bit with cv2.convertScaleAbs, and add it to the original image to obtain the sharpened image. Show the original image, the Laplacian image, the absolute-scaled Laplacian, and the final image in a 2x2 grid.

```
# ================ ( Laplacian Filter ) ====================
# 0. Image file path
imgFile = "moon.jpg"
# 1. Load image and convert to grayscale
img = cv2.imread(imgFile, 0)
# 2. Manually define the Laplacian Kernel
# This is the standard 3x3 kernel that detects horizontal and vertical edges
manual_kernel = np.array([
    [0,  1,  0],
    [1, -4,  1],
    [0,  1,  0]
], dtype=np.float32)
# Alternative kernel: Detects diagonal edges as well
# manual_kernel = np.array([
#     [1,  1,  1],
#     [1, -8,  1],
#     [1,  1,  1]
# ], dtype=np.float32)
# 3. Apply the filter using filter2D
# -1 indicates the output image will have the same depth as the source
# However, for Laplacian, use cv2.CV_64F to avoid clipping negative values
laplacian_manual = cv2.filter2D(img, cv2.CV_64F, manual_kernel)
# 4. Take absolute value and convert back to 8-bit (0-255)
laplacian_abs = cv2.convertScaleAbs(laplacian_manual)
finalImage = img + laplacian_abs
# 5. Display images
plt.figure(figsize=(12, 10))
```

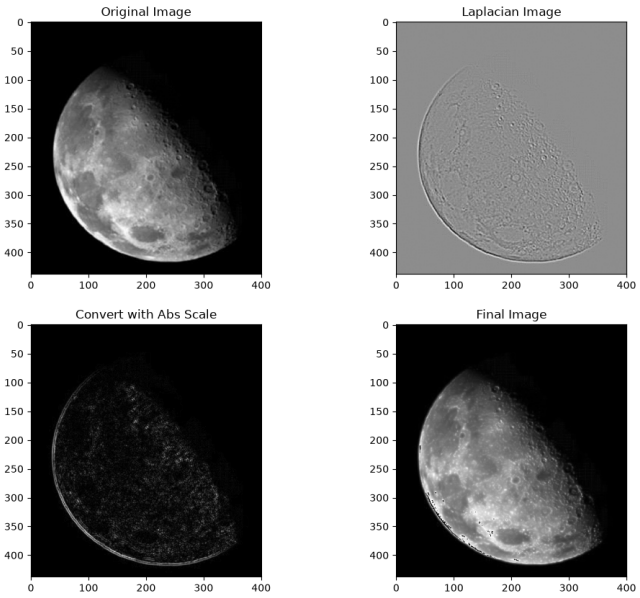
```
plt.subplot(221), plt.imshow(img, cmap='gray'), plt.title('Original Image')
plt.subplot(222), plt.imshow(laplacian_manual, cmap='gray'), plt.title('Laplacian
Image')
plt.subplot(223), plt.imshow(laplacian_abs, cmap='gray'), plt.title('Convert with Abs
Scale')
plt.subplot(224), plt.imshow(finalImage, cmap='gray'), plt.title('Final Image')
plt.show()
```



Procedural Task#2

Repeat the Laplacian sharpening using OpenCV's built-in function. First apply a 3x3 Gaussian blur with cv2.GaussianBlur to reduce noise so that tiny details are not picked up as edges, then apply cv2.Laplacian with cv2.CV_64F to capture both positive and negative gradients, convert the result to 8-bit with cv2.convertScaleAbs, and add it to the original image. Show the original image, the Laplacian image, the absolute-scaled Laplacian, and the final image in a 2x2 grid.

```
# ================== ( Laplacian Filter ) ================
# 1. Optional: Apply Gaussian Blur to reduce noise
# This prevents the algorithm from picking up tiny details as edges
blurred = cv2.GaussianBlur(img, (3, 3), 0)
# 2. Apply the Laplacian Algorithm
# Use CV_64F to capture both positive and negative gradients (essential for edges)
laplacian = cv2.Laplacian(blurred, cv2.CV_64F)
# 3. Convert back to 8-bit image to view it properly
laplacian_8bit = cv2.convertScaleAbs(laplacian)
# 4. Adding Laplace and origianl Image
finalImage2 = img + laplacian_8bit
# 5. Display images
plt.figure(figsize=(12, 10))
plt.subplot(221), plt.imshow(img, cmap='gray'), plt.title('Original Image')
plt.subplot(222), plt.imshow(laplacian, cmap='gray'), plt.title('Laplacian Image')
plt.subplot(223), plt.imshow(laplacian_8bit, cmap='gray'), plt.title('Convert with Abs
Scale')
plt.subplot(224), plt.imshow(finalImage2, cmap='gray'), plt.title('Final Image')
plt.show()
```

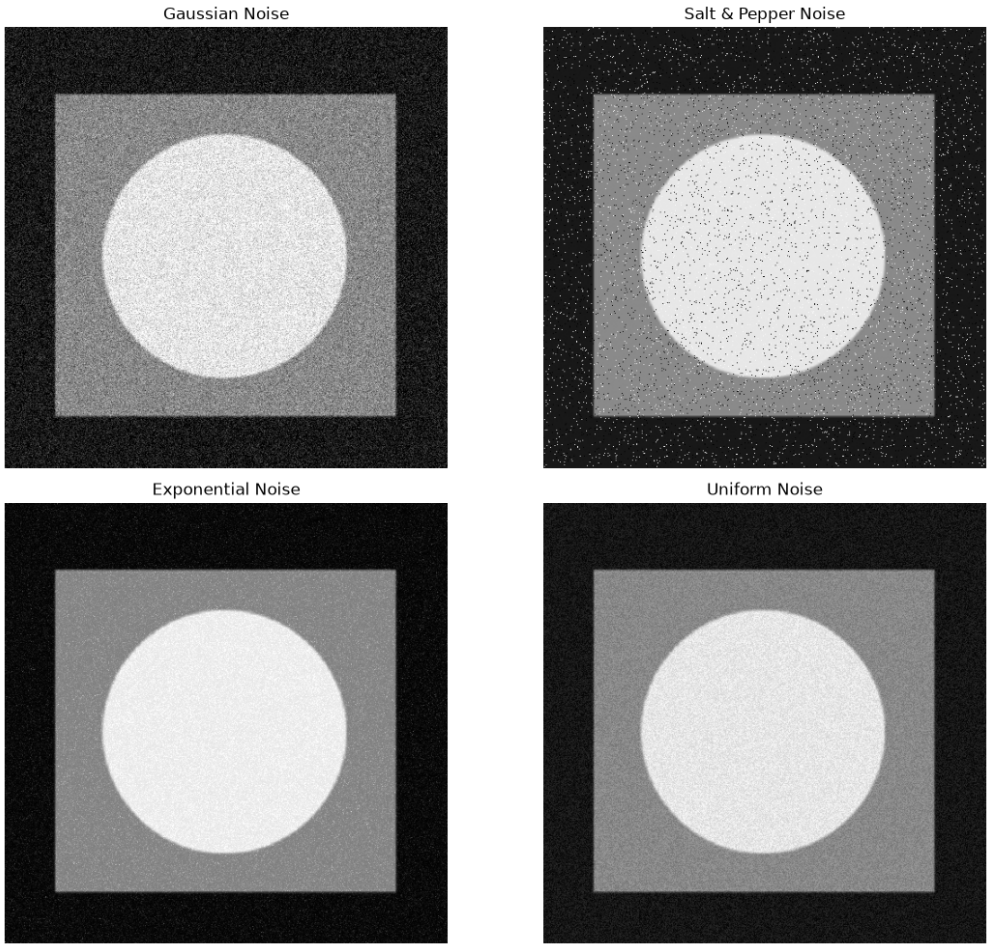


Procedural Task#3

Add four types of noise to a grayscale image. Write an add_noise function that generates Gaussian noise (mean 0, variance 0.01), salt-and-pepper noise (4% of the pixels, half salt and half pepper), exponential noise (scale 10), and uniform noise (range -20 to 20) for noise.jpg, clipping the results to 0-255. Show the four noisy images in a 2x2 grid.

```
# ========== ( Adding nosie to an Image ) =================
# 0. Read an Image
imgPath = 'noise.jpg'
img = cv2.imread(imgPath, 0)
# 1. Function that handle all noise algorithm (Guassian, Exp, Uniform & Salt&Pepper)
def add_noise(image, noise_type):
    row, col = image.shape
    if noise_type == "gaussian":
        mean = 0
        var = 0.01  # Adjust for more/less noise
        sigma = var**0.5
        gauss = np.random.normal(mean, sigma, (row, col))
        noisy = image / 255 + gauss
        return np.clip(noisy * 255, 0, 255).astype(np.uint8)
    elif noise_type == "salt_pepper":
        s_vs_p = 0.5
        amount = 0.04
        noisy = np.copy(image)
        # Salt (white pixels)
        num_salt = np.ceil(amount * image.size * s_vs_p)
        coords = [np.random.randint(0, i - 1, int(num_salt)) for i in image.shape]
        noisy[tuple(coords)] = 255
        # Pepper (black pixels)
        num_pepper = np.ceil(amount * image.size * (1.0 - s_vs_p))
        coords = [np.random.randint(0, i - 1, int(num_pepper)) for i in image.shape]
        noisy[tuple(coords)] = 0
        return noisy
    elif noise_type == "exponential":
        # Scale is the inverse of the rate (lambda)
        expo = np.random.exponential(scale=10.0, size=(row, col))
        noisy = image + expo
        return np.clip(noisy, 0, 255).astype(np.uint8)
    elif noise_type == "uniform":
        low = -20
        high = 20
        uni = np.random.uniform(low, high, (row, col))
        noisy = image + uni
        return np.clip(noisy, 0, 255).astype(np.uint8)
# 2. Load image in grayscale
img = cv2.imread(imgPath, 0)
```

```python
# 3. Generate Noisy Images
noise_gauss = add_noise(img, "gaussian")
noise_sp = add_noise(img, "salt_pepper")
noise_expo = add_noise(img, "exponential")
noise_uni = add_noise(img, "uniform")
# 4. Display in 2x2 Grid
plt.figure(figsize=(12, 10))
plt.subplot(221), plt.imshow(noise_gauss, cmap='gray'), plt.title('Gaussian Noise')
plt.subplot(222), plt.imshow(noise_sp, cmap='gray'), plt.title('Salt & Pepper Noise')
plt.subplot(223), plt.imshow(noise_expo, cmap='gray'), plt.title('Exponential Noise')
plt.subplot(224), plt.imshow(noise_uni, cmap='gray'), plt.title('Uniform Noise')
for i in range(1, 5):
    plt.subplot(220 + i).axis('off')
plt.tight_layout()
plt.show()
```



Procedural Task#4

Apply the arithmetic mean filter with a 3x3 window in two ways. Method 1 uses OpenCV's built-in cv2.blur on the Gaussian-noise image. Method 2 builds the kernel manually as a 3x3 matrix of ones divided by 9 (the number of pixels in the window) and applies it with cv2.filter2D. Display the filtered arrays.

```
# --- METHOD 1: Using Built-in OpenCV function ---
# 2. Apply Arithmetic Mean Filter (using a 5x5 blur window)
# The larger the numbers in (3, 3), the smoother the image becomes.,
# you can change kernel size to (5,5), (7,7) etc.
filtered_img = cv2.blur(noise_gauss, (3, 3))
display(filtered_img)
# --- METHOD 2: Manual implementation using a Kernel ---
# An Arithmetic Mean kernel is just a matrix of 1s divided by the total number of
pixels.
kernel_size = 3
kernel = np.ones((kernel_size, kernel_size), np.float32) / (kernel_size * kernel_size)
arithmetic_mean_manual = cv2.filter2D(img, -1, kernel)
display(arithmetic_mean_manual)
```

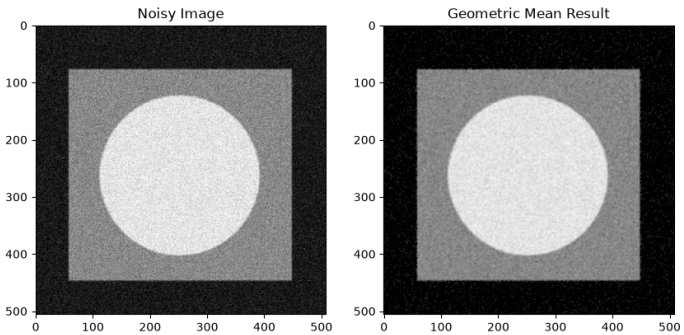
```
array([[30, 36, 34, ..., 15, 26, 32],
       [24, 31, 34, ..., 15, 20, 28],
       [31, 29, 37, ..., 24, 32, 37],
       ...,
       [14, 13, 12, ..., 31, 17, 10],
       [23, 18, 16, ..., 28, 23,  9],
       [30, 22, 21, ..., 24, 22, 12]], shape=(506, 508), dtype=uint8)
```

```
array([[24, 24, 24, ..., 24, 24, 24],
       [24, 24, 24, ..., 24, 24, 24],
       [24, 24, 24, ..., 24, 24, 24],
       ...,
       [24, 24, 24, ..., 24, 24, 24],
       [24, 24, 24, ..., 24, 24, 24],
       [24, 24, 24, ..., 24, 24, 24]], shape=(506, 508), dtype=uint8)
```

Procedural Task#5

Apply the geometric mean filter to the Gaussian-noise image. The geometric mean of a window is the product of its pixels raised to the power 1/(m·n). It is computed by taking the log of the image (adding a tiny epsilon to avoid log(0)), averaging the logs with a 3x3 box filter, and taking the exponent of the result. Show the noisy image and the filtered result side by side.

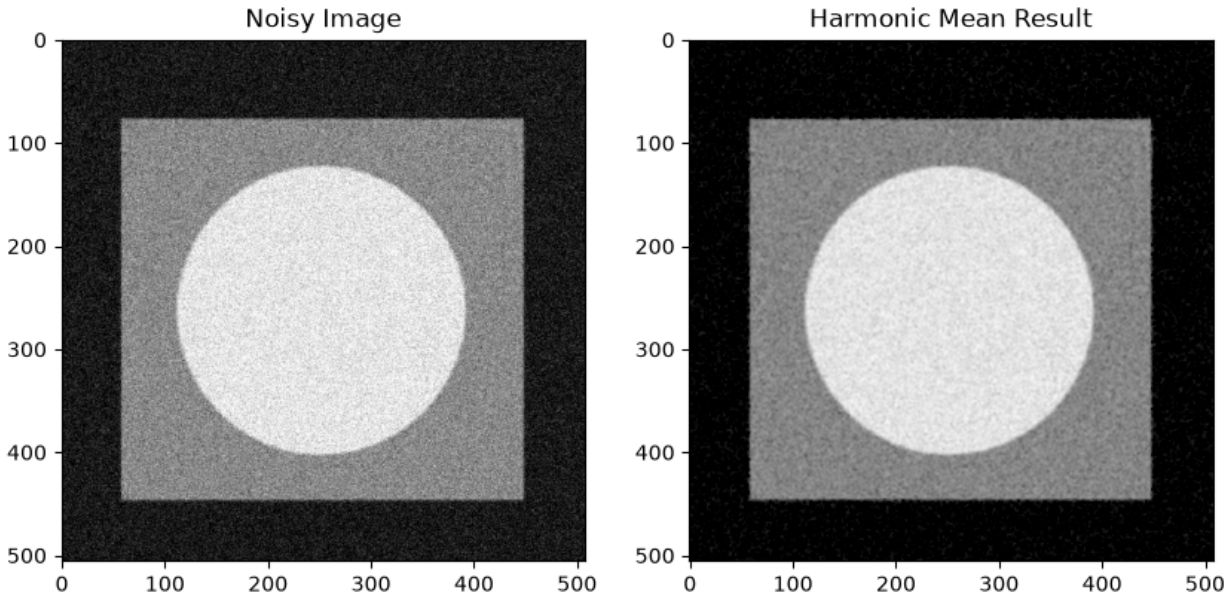
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
def geometric_mean_filter(image, kernel_size=3):
    # 1. Convert to float64 and add a tiny epsilon to avoid log(0)
    img_float = image.astype(np.float64) + 1e-6
    # 2. Apply the log-transform
    log_img = np.log(img_float)
    # 3. Apply a box filter to the log image (calculates the average of logs)
    kernel = np.ones((kernel_size, kernel_size), np.float32) / (kernel_size *
kernel_size)
    mean_log = cv2.filter2D(log_img, -1, kernel)
    # 4. Take the exponent to get back to the geometric mean
    geo_mean = np.exp(mean_log)
    # 5. Clip and convert back to 8-bit
    return np.clip(geo_mean, 0, 255).astype(np.uint8)
# === List of noisy images
# noise_gauss = add_noise(img, "gaussian")
# noise_sp = add_noise(img, "salt_pepper")
# noise_expo = add_noise(img, "exponential")
# noise_uni = add_noise(img, "uniform")
noise_img = noise_gauss
# Apply filter
result = geometric_mean_filter(noise_img, kernel_size=3)
# Display Comparison
plt.figure(figsize=(10, 5))
plt.subplot(121), plt.imshow(noise_img, cmap='gray'), plt.title('Noisy Image')
plt.subplot(122), plt.imshow(result, cmap='gray'), plt.title('Geometric Mean Result')
plt.show()
```



Procedural Task#6

Apply the harmonic mean filter to the Gaussian-noise image. The harmonic mean of a window is its number of pixels divided by the sum of the reciprocals of its pixels. Compute the reciprocal image (with a tiny epsilon to handle black pixels), sum the reciprocals over a 3x3 window with cv2.filter2D, and divide the window size by that sum. Show the noisy image and the filtered result side by side.

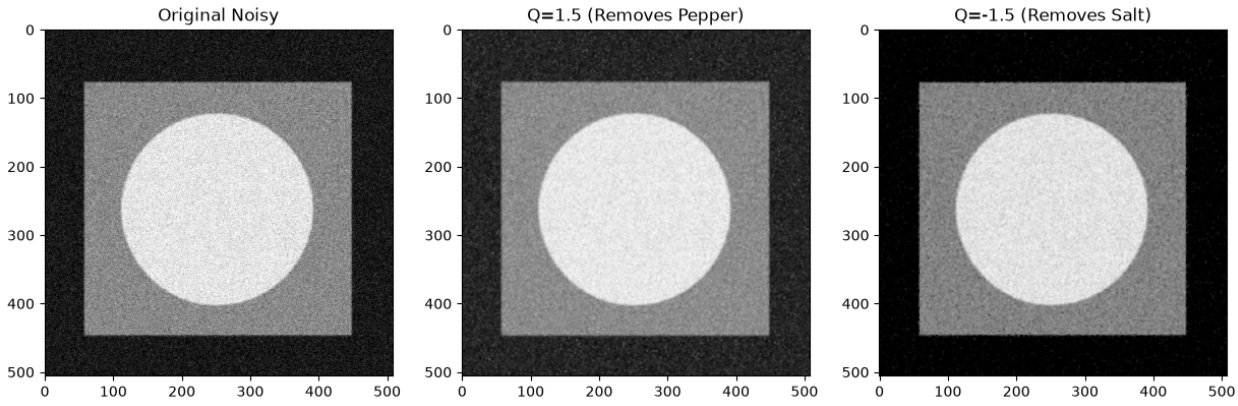
```
def harmonic_mean_filter(image, kernel_size=3):
    # 1. Convert to float64 to avoid overflow during calculation
    # Add a tiny epsilon to handle black pixels (0)
    img_float = image.astype(np.float64) + 1e-6
    # 2. Calculate the reciprocal of every pixel (1/P)
    reciprocal_img = 1.0 / img_float
    # 3. Sum the reciprocals in a window and divide by window size
    # This is equivalent to applying a standard mean filter to the reciprocal image
    kernel = np.ones((kernel_size, kernel_size), np.float32)
    sum_reciprocal = cv2.filter2D(reciprocal_img, -1, kernel)
    # 4. Final step of the formula: (Window Size) / Sum of Reciprocals
    # Note: Window size is kernel_size * kernel_size
    n = kernel_size * kernel_size
    harmonic_mean = n / sum_reciprocal
    # 5. Clip the results and convert back to 8-bit
    return np.clip(harmonic_mean, 0, 255).astype(np.uint8)
# === List of noisy images
# noise_gauss = add_noise(img, "gaussian")
# noise_sp = add_noise(img, "salt_pepper")
# noise_expo = add_noise(img, "exponential")
# noise_uni = add_noise(img, "uniform")
noise_img = noise_gauss
# Apply the filter
cleaned_img = harmonic_mean_filter(noise_img, kernel_size=3)
# Display Comparison
plt.figure(figsize=(10, 5))
plt.subplot(121), plt.imshow(noise_img, cmap='gray'), plt.title('Noisy Image')
plt.subplot(122), plt.imshow(cleaned_img, cmap='gray'), plt.title('Harmonic Mean
Result')
plt.show()
```


Noisy Image
Harmonic Mean Result
0
100
200
300
400
500

Procedural Task#7

Apply the contra-harmonic mean filter to the Gaussian-noise image. For a window, the output is the sum of the pixels raised to the power Q+1 divided by the sum of the pixels raised to the power Q. A positive Q (here 1.5) removes pepper noise, and a negative Q (here -1.5) removes salt noise. Show the noisy image, the Q=1.5 result, and the Q=-1.5 result side by side.

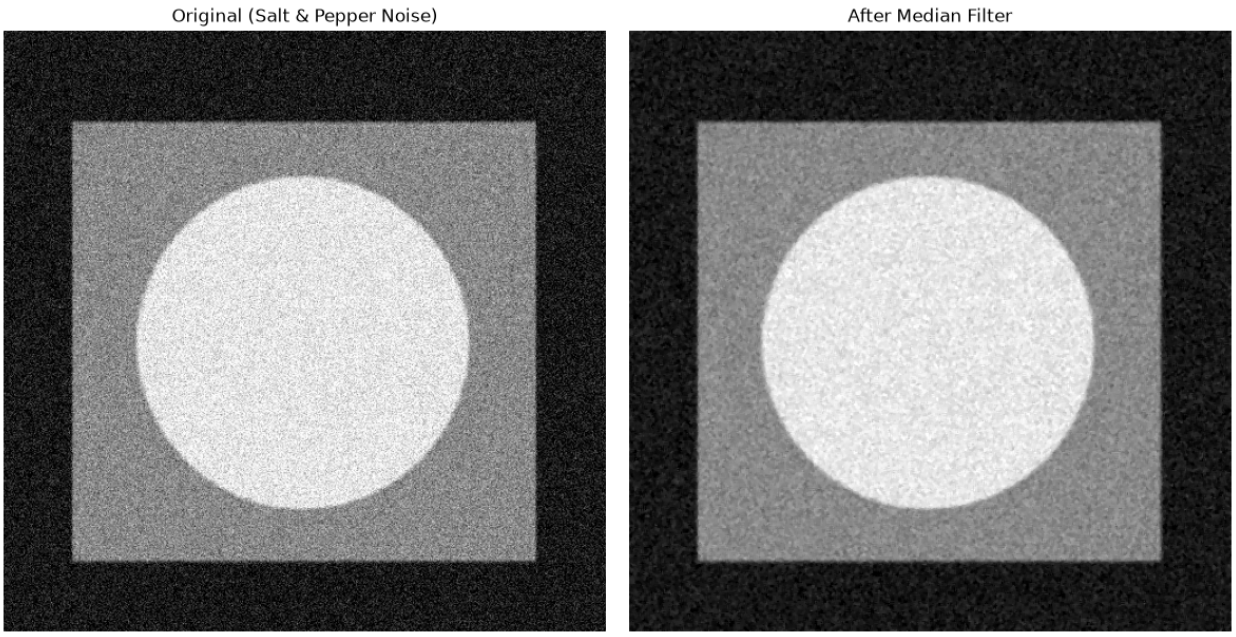
```
def contra_harmonic_mean(image, kernel_size=3, Q=0):
    # 1. Convert to float to handle high powers and prevent overflow
    img_float = image.astype(np.float64) + 1e-6
    # 2. Calculate numerator: Sum of pixels to the power (Q + 1)
    numerator = np.power(img_float, Q + 1)
    kernel = np.ones((kernel_size, kernel_size), np.float32)
    num_sum = cv2.filter2D(numerator, -1, kernel)
    # 3. Calculate denominator: Sum of pixels to the power (Q)
    denominator = np.power(img_float, Q)
    den_sum = cv2.filter2D(denominator, -1, kernel)
    # 4. Divide and clean up
    # Handle potential division by zero
    result = num_sum / (den_sum + 1e-6)
    return np.clip(result, 0, 255).astype(np.uint8)
# === List of noisy images
# noise_gauss = add_noise(img, "gaussian")
# noise_sp = add_noise(img, "salt_pepper")
# noise_expo = add_noise(img, "exponential")
# noise_uni = add_noise(img, "uniform")
noise_img = noise_gauss
# --- APPLICATIONS ---
# Q > 0 (e.g., 1.5): Removes Pepper Noise (black spots)
pepper_removed = contra_harmonic_mean(noise_img, kernel_size=3, Q=1.5)
# Q < 0 (e.g., -1.5): Removes Salt Noise (white spots)
salt_removed = contra_harmonic_mean(noise_img, kernel_size=3, Q=-1.5)
# --- DISPLAY ---
plt.figure(figsize=(15, 5))
plt.subplot(131), plt.imshow(noise_img, cmap='gray'), plt.title('Original Noisy')
plt.subplot(132), plt.imshow(pepper_removed, cmap='gray'), plt.title('Q=1.5 (Removes
Pepper)')
plt.subplot(133), plt.imshow(salt_removed, cmap='gray'), plt.title('Q=-1.5 (Removes
Salt)')
plt.show()
```


Original Noisy
Q=1.5 (Removes Pepper)
Q=-1.5 (Removes Salt)
0
100
200
300
400
500

Procedural Task#8

Apply the median filter to the Gaussian-noise image using cv2.medianBlur with a 3x3 kernel (the kernel size must be an odd integer). Each pixel is replaced by the median of its neighborhood. Show the noisy image and the filtered result side by side.

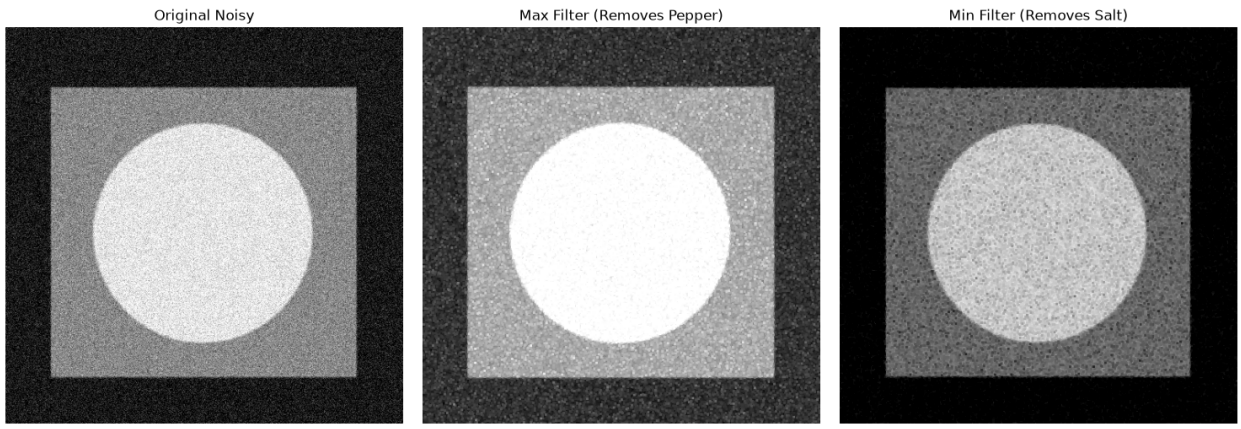
```
# === List of noisy images
# noise_gauss = add_noise(img, "gaussian")
# noise_sp = add_noise(img, "salt_pepper")
# noise_expo = add_noise(img, "exponential")
# noise_uni = add_noise(img, "uniform")
noise_img = noise_gauss
# 2. Apply Median Filter
# The second argument is the 'ksize' (kernel size).
# It MUST be an odd integer (3, 5, 7, etc.)
median_result = cv2.medianBlur(noise_img, 3)
# 3. Compare Results
plt.figure(figsize=(12, 6))
plt.subplot(121), plt.imshow(noise_img, cmap='gray'), plt.title('Original (Salt &
Pepper Noise)'), plt.axis('off')
plt.subplot(122), plt.imshow(median_result, cmap='gray'), plt.title('After Median
Filter'), plt.axis('off')
plt.tight_layout()
plt.show()
```



Procedural Task#9

Apply the max and min filters to the Gaussian-noise image with a 3x3 kernel. The max filter (dilation, cv2.dilate) picks the brightest pixel in each neighborhood and removes pepper noise, while the min filter (erosion, cv2.erode) picks the darkest pixel and removes salt noise. Show the noisy image, the max filter result, and the min filter result side by side.

```
# === List of noisy images
# noise_gauss = add_noise(img, "gaussian")
# noise_sp = add_noise(img, "salt_pepper")
# noise_expo = add_noise(img, "exponential")
# noise_uni = add_noise(img, "uniform")
noise_img = noise_gauss
# 2. Define the Kernel size (e.g., 3x3)
kernel = np.ones((3, 3), np.uint8)
# 3. Apply Max Filter (Dilation)
# This picks the brightest pixel in the neighborhood.
max_filter = cv2.dilate(noise_img, kernel)
# 4. Apply Min Filter (Erosion)
# This picks the darkest pixel in the neighborhood.
min_filter = cv2.erode(noise_img, kernel)
# --- Display Results ---
plt.figure(figsize=(15, 5))
plt.subplot(131), plt.imshow(noise_img, cmap='gray'), plt.title('Original Noisy')
plt.subplot(132), plt.imshow(max_filter, cmap='gray'), plt.title('Max Filter (Removes
Pepper)')
plt.subplot(133), plt.imshow(min_filter, cmap='gray'), plt.title('Min Filter (Removes
Salt)')
for ax in plt.gcf().axes: ax.axis('off')
plt.tight_layout()
plt.show()
```



Procedural Task#10

Apply the midpoint filter to the Gaussian-noise image. The midpoint of a window is the average of its maximum value (dilation) and its minimum value (erosion), computed here with a 3x3 kernel and converted back to 8-bit. Show the noisy image and the filtered result side by side.

```
def midpoint_filter(image, kernel_size=3):
    # 1. Define the structural element (kernel)
    kernel = np.ones((kernel_size, kernel_size), np.uint8)
    # 2. Find the Maximum value in the neighborhood (Dilation)
    max_img = cv2.dilate(image, kernel).astype(np.float64)
    # 3. Find the Minimum value in the neighborhood (Erosion)
    min_img = cv2.erode(image, kernel).astype(np.float64)
    # 4. Calculate the average of Max and Min
    midpoint = (max_img + min_img) / 2
    # 5. Convert back to 8-bit
    return midpoint.astype(np.uint8)
# === List of noisy images
# noise_gauss = add_noise(img, "gaussian")
# noise_sp = add_noise(img, "salt_pepper")
# noise_expo = add_noise(img, "exponential")
# noise_uni = add_noise(img, "uniform")
noise_img = noise_gauss
# Apply the Midpoint Filter
result = midpoint_filter(noise_img, kernel_size=3)
# --- Display Results ---
plt.figure(figsize=(12, 6))
plt.subplot(121), plt.imshow(noise_img, cmap='gray'), plt.title('Original
(Gaussian/Uniform Noise)'), plt.axis('off')
plt.subplot(122), plt.imshow(result, cmap='gray'), plt.title('After Midpoint Filter'),
plt.axis('off')
plt.tight_layout()
plt.show()
```

